PUBLIC DISCLOSURE EDITION

ODRE PQC v0.2.9

Public Technical Whitepaper

English edition

Public edition · 2 September 2026

Contents

Document identity

Product	ODRE PQC
Version	0.2.9
Artifact	odre_pqc-0.2.9-py3-none-any.whl
Wheel SHA-256	A8AFA10EE0AFACEF1C0FAF55FF07B96C722920E5AEE8692F6F026E804F028697
Evidence manifest SHA-256	4AF2C88FDEA0CB1FB030E4AB6D5B41394CC52397664364233482E3EC14945F3B
Runtime cryptography	OpenSSL 3.5.8 / ML-KEM-768 / ML-DSA-65
Validated platforms	Windows Server 2022 AMD64; Ubuntu 24.04.4 LTS AMD64
Final audit verdict	ODRE_PQC_v0.2.9_FULL_DEVSEC_AUDIT_PASS

1. Executive summary

ODRE PQC is an application-embedded post-quantum security boundary for protected server routes. It packages cryptographic runtime management, authenticated session establishment, strict sequencing, replay resistance, protected request and response semantics, native artifact trust, lifecycle control, and fail-closed enforcement behind a small integration surface.

Version 0.2.9 is the sealed cross-platform release candidate identified by the SHA-256 in this document. The exact same wheel passed a fresh full development-security audit on Windows Server 2022 AMD64 and Ubuntu 24.04.4 LTS AMD64. No new confirmed Critical, High, or Medium product finding was produced within the executed scope.

This paper describes evidence, not absolute security. PASS applies only to the named artifact, platforms, threat models, configurations, and tests. Production deployment remains a separate operator-controlled decision.

2. Product purpose and scope

The product protects selected application routes rather than replacing TLS, identity, authorization, secure coding, patch management, host hardening, backups, or monitoring. It is designed as an additional application security boundary: protected business handlers run only after the PQC boundary accepts the request.

The public integration model is intentionally small: initialize and verify the product, register it with the application using install(app), start the server, and inspect authenticated runtime status. Platform differences remain inside the product adapter and native runtime bundles.

Integration model

```
odre-pqc init
odre-pqc doctor
odre-pqc verify
from odre_pqc import install
install(app)
server start → odre-pqc status
```

3. Architecture

The boundary consists of an application integration layer, admission and semantic-binding controls, a hybrid PQC handshake and session layer, replay and sequence state, protected request/response envelopes, platform-specific native trust controls, and authenticated lifecycle/status reporting.

The wire contract is versioned as odre-pqc/3. Protected request semantics observed by the handler are the authenticated inner semantics. Protected response status, ordered headers, cookies, and body remain inside the protected response frame; outer transport metadata is restricted.

The distribution uses one Python wheel with platform-specific runtime bundles. The sealed package binds package metadata, manifests, native archives, and integrity checks to an exact release identity.

4. Cryptographic profile

The audited runtime identifies as OpenSSL 3.5.8 (OpenSSL_version_num 0x30500080). Real ML-KEM-768 and ML-DSA-65 self-tests and the hybrid handshake passed on both target operating systems.

ODRE PQC does not silently fall back to classical-only protection for a protected route. If the provider, key material, session state, replay state, native trust, or runtime health cannot satisfy the contract, the protected route is blocked before the business handler runs.

5. Security contract

The core contract is fail-closed. Invalid or ambiguous inputs, replay or sequence violations, cross-session or cross-route mismatch, unauthenticated status, unsafe native objects, initialization failures, and unsafe lifecycle transitions must not be converted into successful protected execution.

Authenticated request semantics equal handler-observed semantics. Protected response semantics remain confidential and integrity-protected. Security-sensitive OS executables and native libraries are bound to trusted objects. Runtime ON status is authenticated to a live instance and fresh state. Security resources outlive every admitted protected request.

6. Threat model and trust boundaries

The audit exercised network-originated tampering, replay and concurrency, malformed and oversized inputs, semantic confusion, response metadata leakage, crash/restart state, local filesystem replacement, symbolic-link or reparse manipulation, dependency and provider injection, and lifecycle races.

The product assumes the underlying operating system, CPU, Python runtime, and authorized administrator are not fully compromised. It does not claim protection against a malicious kernel, unrestricted administrator control, physical extraction, compromised application logic after successful admission, denial of service outside tested controls, or future cryptanalytic breaks.

7. Verification and audit result

The sealed wheel was installed from a fresh copy under hash lock. Start and end hashes were identical. Earlier reports were not reused as PASS evidence, and the product bytes were not changed during the audit.

Static review covered 38 Python files and 7,160 lines, package metadata, RECORD, platform support data, both native manifests, and both runtime archives. Wheel RECORD verification passed for 45 of 45 hashed entries. Package contamination was zero. Fresh advisory queries covered 19 Python dependencies with zero query errors and zero vulnerability records returned at audit time.

Six issues were found in the audit harness, not the product. Each harness issue was corrected, the affected gate was rerun, and the sealed product wheel remained unchanged.

Published audit metrics

Metric	Windows	Ubuntu
Platform verdict	PASS	PASS
Replay-race requests	1,280	2,880

Double handler execution	0	0
Native load attempts	3,000	-
Atomic replacements observed	1,062	-
Junction create/remove cycles	3,000	-
Late-promotion parent swaps	-	600 / 600
Untrusted load / misbinding	0	0
Unresolved Critical / High / Medium	0 / 0 / 0	0 / 0 / 0

8. Cross-platform results

Windows verification included fresh initialization, second-init protection, doctor, verify, status, session transition, replay race, request/response binding, ACL trust, authenticated status, shutdown resource lifetime, protocol tamper, crash recovery, and native load stress.

Ubuntu verification included fresh initialization, negative diagnostics, real PQC, replay race, admission fuzzing, malformed and oversized input, HTTP smuggling shapes, response secrecy, WebSocket fail-closed behavior, native object trust, concurrent initialization, worker collision, graceful and forced restart, and late-promotion parent-object races.

9. Release discipline and evidence

ODRE PQC uses immutable release identity. An audited artifact is named by version and SHA-256. If bytes change, a new artifact and new verification cycle are required; an earlier version is not silently reissued under the same identity.

The evidence set separately records product findings, harness issues, and environment issues. For v0.2.9: discovered product findings 0, remediated product findings 0, environment issues 0, unresolved Critical/High/Medium 0. Production network contact was 0 and production was not changed.

10. Deployment guidance

Operators should verify the wheel SHA-256 before installation, use the supported OS and Python baseline, run init, doctor, and verify in a fresh controlled environment, register install(app), start the application, and confirm authenticated status before enabling protected traffic.

Deployment must also include independent TLS configuration, authorization, application security review, secret handling, host patching and hardening, logging, monitoring, backups, rollback planning, and change control. A successful development-security audit is not a substitute for a production risk assessment.

11. Public disclosure boundary

This public paper intentionally excludes private keys, secrets, production topology, customer data, exact internal paths, operational identifiers, exploit scripts, full harness source, and procedural detail that would materially reduce attack cost. It includes enough architecture, security contract, artifact identity, test scope, metrics, and limitations for external technical evaluation.

12. Conclusion

ODRE PQC v0.2.9 is a sealed, cross-platform, application-embedded PQC security boundary whose exact wheel passed the stated full development-security audit on both official operating systems. The strongest claim supported by the evidence is scoped and reproducible: the named artifact satisfied the named gates under the named test conditions with no confirmed unresolved Critical, High, or Medium product finding.

Artifact verification

SHA-256

```
A8AFA10EE0AFACEF1C0FAF55FF07B96C722920E5AEE8692F6F026E804F028697
```

Evidence manifest SHA-256

```
4AF2C88FDEA0CB1FB030E4AB6D5B41394CC52397664364233482E3EC14945F3B
```

Verification command

```
python -m pip hash odre_pqc-0.2.9-py3-none-any.whl
```

Publication note

Prepared from the sealed v0.2.9 full development-security audit record. Public claims are deliberately scoped to the exact artifact and executed evidence. ODRE AI · Korea · 2026